



Universität Regensburg

Fakultät für Sprach-, Literatur- und Kulturwissenschaften

Bachelorarbeit

Im Studiengang Englische Sprachwissenschaft

Sommersemester 2026

Can AI Replace Traditional Textbooks?

A Comparative Corpus Analysis of LLM-Generated vs. Textbook Example Sentences

Autor:	Simon John
Matrikelnummer:	2175939
Adresse:	Wildbachweg 5, 93059 Regensburg
E-Mail:	simon.john@stud.uni-regensburg.de
Studiengang:	BA Englische Sprachwissenschaft
Abgabedatum:	April 16, 2026
Gutachter:	Dr. Thorsten Brato

Abstract

Can Large Language Models (LLMs) generate pedagogical input that is both register-appropriate and systematically different from traditional textbook materials? This is the question this study seeks to answer. The Natural Reference Corpus (NRC, $N = 5,483$) compiled from B1/B2 English as a foreign language (EFL) textbooks was contrasted with the Synthetic Control Corpus (SCC, $N = 1,237$), which was artificially generated by Google Gemini 2.5 Flash under the following register conditions: HIGH (formal workplace setting), NEUTRAL (instructional setting) and LOW (casual, conversational setting). For this study, the two corpora were filtered specifically for sentences containing ten selected high-frequency, highly polysemous verbs such as *run*, *take* and *make*. The verbs in these sentences were then classified as either LITERAL or IDIOMATIC via an LLM-as-a-judge protocol, which was subsequently validated through inter-rater reliability (IRR) testing ($\kappa = 0.62$). Statistical analysis confirmed three hypotheses. First, sentences generated by Artificial Intelligence (AI) yielded significantly more IDIOMATIC contexts than the textbook sentences ($\chi^2 = 143.90, p < 0.001$). Second, the NRC aligned most closely with the NEUTRAL subsection of the SCC ($V = 0.024$), hinting at a high use of a distinct instructional register in English textbooks. Third, rather than behaving uniformly, idiomaticity was lemma-selective in both corpora. The findings suggest that, if prompted explicitly, LLMs can approximate or even exceed textbook register behaviour. This “naturalness”, however, depends heavily on communicative framing, rather than on model capability alone. Implications for future textbook design and AI-assisted pedagogy are discussed.

Contents

1	Introduction	1
1.1	Background & Motivation	1
1.2	Research Gap & Study Rationale	1
1.3	Research Question & Design Overview	2
1.4	Scope & Structure of the Thesis	3
2	Theoretical Framework	3
2.1	Register Theory: The Anatomy of “Casualness”	4
2.1.1	The Concept of Register	4
2.1.2	Biber’s Multi-Dimensional Analysis (MDA)	4
2.1.3	Linguistic Markers of Involvement	5
2.1.4	Grammatical Mood & Register	6
2.2	Phraseology & Idiomaticity	7
2.2.1	Sinclair’s Dual Framework	7
2.2.2	The Open-Choice Principle	7
2.2.3	The Idiom Principle	7
2.2.4	De-lexicalisation in High-Frequency Verbs	7
2.2.5	The LLM Phraseology Paradox	7
2.3	Cognitive Complexity & Pedagogical Input	7
2.3.1	Krashen’s Input Hypothesis ($i + 1$)	7
2.3.2	Sweller’s Cognitive Load Theory (CLT)	7
2.3.3	The Paradox of Simplification	7
2.4	Synthesis: Evaluating Synthetic Input	7
3	Methodology	8
3.1	Research Design & Objectives	8
3.1.1	Study Design Overview	8
3.1.2	Research Question & Hypotheses	8
3.2	Corpus Compilation	9
3.2.1	Natural Reference Corpus (NRC)	9
3.2.2	Synthetic Control Corpus (SCC)	9
3.2.3	Pilot Study & Prompt Refinement	9
3.2.4	Reproducibility Specifications	9
3.3	Data Processing Pipeline	9
3.3.1	POS-Tagging & Syntactic Disambiguation	9
3.3.2	Semantic Classification	9

3.3.3	Data Quality Assurance	9
3.3.4	Inter-Rater Reliability (IRR)	9
3.4	Statistical Analysis	10
3.4.1	Comparison Strategy.....	10
3.4.2	Inferential Testing & Thresholds.....	10
3.4.3	Statistical Procedure & Frequency Normalisation.....	10
3.5	Ethical Considerations & Data Use	10
4	Results	10
4.1	Descriptive Overview of the Final Datasets	10
4.2	Overall Corpus Comparison (Testing Hypothesis 1)	10
4.3	Register Effects & Prompt Sensitivity (Testing Hypothesis 2)	11
4.4	Lemma-Level Selectivity (Testing Hypothesis 3)	11
5	Discussion	12
5.1	Summary of Key Findings	12
5.2	Interpreting H1: The Pedagogical Simplification Hypothesis	12
5.3	Interpreting H2: Textbooks as Instructional Register	12
5.4	Interpreting H3: Lemma-Level Selectivity	12
5.5	The V-Shaped Pattern: Register Sensitivity in LLMs	12
5.6	Methodological Reflections	12
5.7	Limitations	12
5.8	Future Research	12
6	Conclusion	12
6.1	Research Summary	12
6.2	Theoretical Contributions	12
6.3	Practical Implications	12
6.4	Final Reflection	12
	References	13
	Documentation of AI Tools Used.....	15
	Plagiarism Statement	16
A	TEC Source Textbooks (B1/B2 Subset).....	17
B	SCC Pilot Study Visualisations.....	18
B.1	Short Interpretation: <i>run</i>	19
B.2	Short Interpretation: <i>take</i>	19

C	Final Dataset Visualisations	21
D	Python Code Documentation	23
D.1	debug.py	23
D.2	scalable_context_generator.py	23
D.3	target_words.txt	27
D.4	sketch_engine_converter_nltk.py	27
D.5	pos_taggerV2NLTK.py	30
D.6	pos_tagger_nltk_for_csv.py	31
D.7	semantic_analyzer.py	33
D.8	irr_sheet_generator.py	37
D.9	compute_split_irr.py	41
D.10	irr_split_results.csv	46
D.11	visualizer.py	46
E	Full AI Documentation	50
E.1	Gemini 2.5 Flash: SCC Generation Prompts	50
E.2	Gemini 2.5 Flash: Semantic Classification Prompts	50
E.3	Gemini 3 Pro: Ideation Prompt	51

List of Tables

1	Framework Summary	7
2	NRC Lemma Distribution	9
3	IRR Confusion Matrix.....	10
4	Final Dataset Overview	10
5	Overall NRC–SCC Contingency.....	10
6	Pairwise Register Comparisons	11
7	Lemma-Level Idiomaticity	11
8	AI Tools Used	15

List of Figures

1	SCC Pilot: Aggregate by Register	18
2	SCC Pilot: Idiomaticity Heatmap	19
3	Final Dataset: Aggregate by Register.....	21
4	Final Dataset: Idiomaticity Heatmap	21
5	Gemini 3 Screenshot 1.....	51
6	Gemini 3 Screenshot 2.....	52
7	Gemini 3 Screenshot 3.....	52
8	Gemini 3 Screenshot 4.....	52
9	Gemini 3 Screenshot 5.....	53

List of Abbreviations

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
CEFR	Common European Framework of Reference for Languages
CLT	Cognitive Load Theory
CSV	Comma Separated Values
EFL	English as a Foreign Language
ETL	Extract, Transform, Load
HIGH	High Register Condition
IDIOMATIC	Idiomatic Usage Label
IRR	Inter-Rater Reliability
JSON	JavaScript Object Notation
LLM	Large Language Model
LITERAL	Literal Usage Label
LOW	Low Register Condition
MDA	Multi-Dimensional Analysis
NEUTRAL	Neutral Register Condition
NLP	Natural Language Processing
NLTK	Natural Language Toolkit
NONE	Invalid Context
NRC	Natural Reference Corpus
POS	Part-of-Speech
SCC	Synthetic Control Corpus
TEC	Textbook English Corpus
QA	Quality Assurance

1 Introduction

1.1 Background & Motivation

Recently, the integration of Large Language Models (LLMs) into the language learning classroom is heavily reshaping how pedagogical content can be produced and distributed. In practical terms, this means that language learners are no longer limited to studying with example texts in textbooks, but can request context-specific alternatives on demand. While this development creates new opportunities for adaptive learning, it also reopens the discussion about a persistent question in applied linguistics: how closely do pedagogical examples resemble authentic language use in everyday communication?

This question is significant because traditional “Textbook English” has long been criticised for being overly stiff and unnatural (Conti and Smith 2016). Traditional learning materials tend to be explicit in meaning and syntactically strict, presenting a “saniti(s)ed” (Curdt-Christiansen and Weninger 2015) version of English that rarely matches how speakers actually interact with each other. Here, Generative Artificial Intelligence (AI) offers a promising solution by generating “natural, casual” English when prompted accordingly. The question is, however, whether LLMs can truly recreate the phraseological nuances of authentic speech, or if they simply produce a statistically average imitation that is just as artificial and “sanitised” as the textbooks they aim to replace.

This tension between teachability and authenticity is not only about style. If learning materials overwhelmingly contain explicit grammatical and syntactical transparency over natural phraseology, learners receive input that is clear, but does not represent how language in different social contexts works. This is bound to cause intelligibility issues in real-world situations outside the English as a foreign language (EFL) classroom. Register-oriented (Biber 1988; Le Foll 2021b) and phraseological research (Sinclair 1991) therefore provide an appropriate basis for an empirical study on this issue.

1.2 Research Gap & Study Rationale

Most recent work regarding AI in education focuses on the technology as an evaluator, assistant or interactional partner for learner output (Holmes et al. 2019). In contrast, comparatively little research has been conducted on AI as a robust

provider of pedagogical input. This is the central gap this thesis addresses.

The secondary gap is methodological. Claims that AI output either sounds “natural” or “unnatural” are often made intuitively without actual testing through objective corpus linguistics procedures. To move on from subjective intuition to quantitative evidence, this thesis operationalises authenticity through measurable linguistic dimensions, specifically focusing on register distribution (Biber 1988) and patterns in idiomaticity (Sinclair 1991).

From the perspective of language acquisition, the relevance of this gap is very practical. The input quality has a direct impact on what learners can notice and acquire over time (Krashen 1985). Even though this study does not measure cognitive load directly, it acknowledges that wording, phraseology and information packaging play a fundamental role in cognitive processing in educational contexts (Spiro et al. 1992; Sweller 1988).

1.3 Research Question & Design Overview

This study uses a data-driven approach, utilizing methods of Corpus Linguistics to address the following research question: *To what extent can Large Language Models (specifically Gemini 2.5 Flash) generate linguistically distinct registers (formal, instructional and casual) that quantitatively differ from human-authored textbook materials?* To answer this question, this study takes an existing Natural Reference Corpus (NRC), compiled by Le Foll (2021a) from EFL textbook materials and compares it to a Synthetic Control Corpus (SCC). The SCC is generated by Google’s Gemini 2.5 Flash Application Programming Interface (API) via role-prompting across three distinct registers: formal workplace language (HIGH register: questions and professional contexts), instructional language (NEUTRAL register: imperatives, directives and rules) and casual conversational language (LOW register: statements in informal contexts). The comparative nature of this study enables testing for alignment between textbook English and the AI generated formal, instructional or casual registers. It might also reveal, if textbooks have a distinct pedagogical register unlike any of these.

In analytical terms, the comparison between the two corpora is made on sentence level, focusing on ten high-frequency, polysemous verbs (e.g., *run*, *take*, *make*). As such verbs are both common and semantically flexible (Stoeckel 2019), research of their occurrences in literal versus idiomatic contexts will provide a quantifiable metric for phraseological authenticity. By tracking how these verbs shift between

transparent, literal meanings and de-lexicalised, idiomatic chunks, the analysis will reveal whether pedagogical materials artificially suppress idiomaticity in favour of grammatical and lexical clarity, and whether the LLM replicates or diverges from this pattern across the generated registers.

1.4 Scope & Structure of the Thesis

The research question is deliberately limited to the phraseology of verbs at the intermediate learner level (B1/B2) for three methodological reasons. First, high-frequency, polysemous verbs have high variability in different contexts within controlled lexical sets (e.g., tracking a single lemma like *run* as it shifts between physical movement and de-lexicalised usage like *run out of time*). Second, intermediate-level material represents the turning point where formulaic competence and idiomaticity become relevant to develop a better understanding of a language (Schmale 2022). Third, restricting the analysis to ten hand-picked lemmas enables intensive profiling of the different registers while still maintaining computational feasibility.

The remainder of the thesis is structured as follows. Chapter 2 establishes a necessary theoretical framework that will provide a valid anchor for the analysis and its interpretation. This chapter focuses on Biber’s Register Theory, Sinclair’s Idiom Principle and cognitive approaches to language input. Chapter 3 outlines the methodology of the analysis, including the compilation of the NRC and SCC. It also explains the Python-based ETL workflow and the LLM-as-a-judge evaluation system. Chapter 4 continues with the presentation of the results of the corpus comparison, which are then interpreted and evaluated against the study’s hypotheses in Chapter 5. Finally, Chapter 6 summarises the findings, acknowledges methodological limitations and discusses implications for the design of future learning materials.

2 Theoretical Framework

This chapter sets a theoretical anchor used to evaluate the AI-generated example sentences against textbook input. Rather than portraying AI output as globally “good” or “bad”, the framework makes the quality of the input measurable through three dimensions: register appropriateness, phraseological naturalness and cognitive manageability. These dimensions are used to make relevant claims about authenticity that can practically be tested by corpus analysis.

2.1 Register Theory: The Anatomy of “Casualness”

2.1.1 The Concept of Register

In linguistics, register refers to the variation of language in situational contexts: who is communicating, for what purpose and under which communicative constraints (Biber et al. 1999). This is significantly different from the genre of a text, with which register is often confused. Genre typically refers to socially recognised types of text (e.g., newspaper article, novel, research paper) and their conventional structure. Fundamentally, genre tells us what kind of text we are looking at, while register tells us how language is produced under those specific circumstances.

This difference is important for the present thesis. If AI-generated content is evaluated without regard to the context and the prompt with which the output has been generated, judgements on its “naturalness” are not valid. A sentence that is totally normal in a formal instructional setting might be inappropriate in a casual conversation, and vice versa. Therefore, this analysis is based on a register-relative approach rather than making absolute claims.

Textbook English has a distinct instructional register that systematically differs from non-pedagogical registers (Le Foll 2021b). Le Foll’s studies in EFL textbooks prove that textbook English’s linguistic profile is optimised for pedagogical transparency rather than authenticity and real-world usability. This distinction is even further analysed in her multi-dimensional study of textbook English as a pedagogical variety (Le Foll 2024).

2.1.2 Biber’s Multi-Dimensional Analysis (MDA)

The central theoretical basis for this chapter is Biber’s (1988) multidimensional account of variation. Biber’s core argument is that linguistic features do not occur randomly. Instead, they occur in patterns that fulfil communicative functions. By doing factor analysis on large corpora, Biber identified different dimensions of variation that can be interpreted functionally across spoken and written contexts.

For the present study, Dimension 1 (Involved vs. Informational Production) is especially relevant. On the involved side, language tends to be interactional, context-dependent and person-oriented. On the informational side, language tends to be dense, explicit and noun-heavy. Textbook examples often lean toward informational clarity by design: explicit lexical choices, full clause structures and decontextualised propositions. To communicate appropriately, however, learners need to

develop competence in involved usage, especially for informal and conversational settings.

This is one of the key motivations for this study. Can AI be used as a supplement for textbook examples, generating input that closely mirrors real-life conversation in order to familiarise learners with authentic language? If role-prompted LLM output is genuinely sensitive to different registers, the HIGH, NEUTRAL and LOW SCC conditions should show measurable changes in behaviour along involvement-related features. Artificially generated output should therefore not collapse into a textbook-like profile.

2.1.3 Linguistic Markers of Involvement

To make “casualness” more tangible, the present study isolates three concrete markers derived from Biber’s register studies (1988; 1999). Rather than recreating Biber’s entire multidimensional model, these markers are operationalised as robust indicators that can be compared across different corpora.

The first marker is the density of first- and second-person pronouns (e.g., *I, we, you*). Involved discourse is typically interpersonal: speakers address themselves and their interlocutors directly. Biber (Biber 1988) links this feature to interpersonal focus and interactional orientation. In spoken English, person deixis is therefore a structural signal of communicative stance (Biber et al. 1999). Textbook examples tend to minimise this interpersonal discourse in favour of explicit third-person or generic formulations. This design is specifically used to support pedagogical clarity, but may under-represent how participants are addressed in natural conversation. Consequently, a high density of interactive pronouns is treated as a key indicator of involved, casual register.

The second marker is spatial and temporal deictic anchoring: references that depend on the space and time of the context (e.g., *here, there, now, then, tomorrow*). Biber describes these forms as “situation-dependent references” (1988). In practice, these anchors help speakers coordinate attention and action while talking. In contrast, pedagogical examples are frequently designed as context-independent utterances (e.g., generic routines, logical truths). Because of their independence from deixis, these utterances can be taken out of context and still make perfect sense.

The third marker is structural reduction. Language that is produced in spontaneous interaction is regularly compressed to optimise speech economy. This can happen

through contractions, ellipses and optional deletions (e.g., complementiser omission) (Biber et al. 1999). Such reductions emerge because conversation relies on shared context, processing economy and speed of the interaction. Conversely, textbooks often suppress reduction in order to teach standard English through maximal transparency. What textbooks tend to ignore are non-standard forms of English that are extremely common in natural conversation. Even though full forms may reduce ambiguity for learners, they also diverge from typical spoken realisations. For this reason, instances of reduction are treated as meaningful indicators for conversational authenticity.

While tracking these markers provides valuable insights into register variation, a comprehensive analysis of pronoun density, deixis and structural reduction falls outside the scope of this thesis and remains open for future research on this topic. Instead, this study approaches the formal/casual level of the English language through an extensive study of phraseology. The theoretical basis for this approach is Sinclair's (1991) distinction between the Open-Choice Principle and the Idiom Principle discussed in section 2.2.

2.1.4 Grammatical Mood & Register

Register and mood are two parameters that have to be distinguished clearly. On the one hand, register refers to the situational context shaped by communicative situation, purpose and participants (Biber 1993). On the other hand, mood is a grammatical category on clause-level that is used as resource to express modality, speech act and interpersonal such as declaratives, interrogatives and imperatives roles (Quirk et al. 1985). While there is a strong correlation between certain moods and registers (e.g., questions are more commonly used in formal contexts, imperatives in instructional contexts and declaratives in casual contexts), it is important to note that the two parameters are not the same. In other words, register is used on the level of the text as a whole, while mood is a feature that can change from one sentence to another within the same text.

Imperatives are essential to instructional and directive language because they encode the performance of actions directly (Kia 2018).

2.2 Phraseology & Idiomaticity

2.2.1 Sinclair’s Dual Framework

2.2.2 The Open-Choice Principle

2.2.3 The Idiom Principle

2.2.4 De-lexicalisation in High-Frequency Verbs

2.2.5 The LLM Phraseology Paradox

2.3 Cognitive Complexity & Pedagogical Input

2.3.1 Krashen’s Input Hypothesis ($i + 1$)

2.3.2 Sweller’s Cognitive Load Theory (CLT)

2.3.3 The Paradox of Simplification

2.4 Synthesis: Evaluating Synthetic Input

Table 1 summarises how these theoretical perspectives map onto the empirical analysis.

Table 1: Theoretical Framework Summary

Dimension	Theoretical Basis	Theoretical Indicator
Register ¹	Biber (1988)	Pronouns, deixis, contractions
Phraseology	Sinclair (1991)	LITERAL vs. IDIOMATIC
Pedagogy	Krashen (1985)	B1/B2 level appropriateness
Processing ¹	Sweller (1988)	Cognitive-load

¹While these indicators define the theoretical boundaries of the constructs, the present empirical pipeline (Chapter 3) restricts its quantitative operationalisation exclusively to the phraseological dimension.

3 Methodology

3.1 Research Design & Objectives

3.1.1 Study Design Overview

3.1.2 Research Question & Hypotheses

The core research question asks: *To what extent can Large Language Models (specifically Gemini 2.5 Flash) generate linguistically distinct registers (formal, instructional and casual) that quantitatively differ from human-authored textbook materials at the level of phraseological behaviour?*

This overarching question is operationalised through three testable hypotheses:

- **H1 (Pedagogical Simplification):** The NRC will show a higher proportion of literal uses than the SCC overall, reflecting pedagogical preference for Open-Choice transparency over Idiom-Principle patterning.
- **H2 (Corpus-Informed Pedagogy):** The NRC will pattern most closely with SCC-NEUTRAL, indicating that textbook English functions as a distinct instructional register rather than approximating formal or casual native discourse.
- **H3 (Controlled Idiomaticity):** The NRC will show selective idiomaticity (non-uniform idiomatic behaviour across verb types) rather than uniformly high or uniformly low idiomaticity, reflecting strategic rather than blanket phraseological simplification.

Table 2: Distribution of target verb lemmas in the NRC dataset after POS filtering ($N = 5,501$). This table reports pre-semantic-classification counts. Following semantic classification, 18 unresolved items were excluded, yielding the final analysis sample of $N = 5,483$ reported in Chapter 4.

Lemma	n	%
<i>run</i>	218	3.96
<i>take</i>	1,067	19.40
<i>make</i>	1,493	27.14
<i>keep</i>	244	4.44
<i>hold</i>	145	2.64
<i>break</i>	137	2.49
<i>drop</i>	32	0.58
<i>turn</i>	290	5.27
<i>see</i>	1,030	18.72
<i>look</i>	845	15.36
Total	5,501	100.00

3.2 Corpus Compilation

3.2.1 Natural Reference Corpus (NRC)

3.2.2 Synthetic Control Corpus (SCC)

3.2.3 Pilot Study & Prompt Refinement

3.2.4 Reproducibility Specifications

3.3 Data Processing Pipeline

3.3.1 POS-Tagging & Syntactic Disambiguation

3.3.2 Semantic Classification

3.3.3 Data Quality Assurance

3.3.4 Inter-Rater Reliability (IRR)

The full agreement distribution is shown in Table 3.

		AI Model (Gemini)		
		LITERAL	IDIOMATIC	Total
Human Rater	LITERAL	62	20	82
	IDIOMATIC	15	89	104
	Total	77	109	186

Table 3: Confusion Matrix of Semantic Classifications for the IRR sample ($N = 186$). The diagonal cells represent raw agreement (81.2%), while the off-diagonal cells represent disagreements penalized by the Cohen’s Kappa calculation ($\kappa = 0.62$).

3.4 Statistical Analysis

3.4.1 Comparison Strategy

3.4.2 Inferential Testing & Thresholds

3.4.3 Statistical Procedure & Frequency Normalisation

3.5 Ethical Considerations & Data Use

4 Results

4.1 Descriptive Overview of the Final Datasets

Table 4: Descriptive overview of final annotated datasets

Condition	N	LITERAL n (%)	IDIOMATIC n (%)
NRC	5,483	2,503 (45.7)	2,980 (54.3)
SCC-HIGH	427	56 (13.1)	371 (86.9)
SCC-NEUTRAL	385	197 (51.2)	188 (48.8)
SCC-LOW	425	81 (19.1)	344 (80.9)
SCC (combined)	1,237	334 (27.0)	903 (73.0)
Combined total	6,720	2,837 (42.2)	3,883 (57.8)

4.2 Overall Corpus Comparison (Testing Hypothesis 1)

Table 5: Overall NRC vs. SCC contingency table (Corpus $\times$ Usage Category)

Corpus	LITERAL n (%)	IDIOMATIC n (%)	Total
NRC	2,503 (45.7)	2,980 (54.3)	5,483
SCC (combined)	334 (27.0)	903 (73.0)	1,237

Table 6: Pairwise NRC vs. SCC register comparisons

Comparison	Idiomatic rate (%)	$\chi^2(1)$ (p)	Cramér’s V
NRC vs. HIGH	54.3 vs. 86.9	170.81 ($p < 0.001$)	0.170
NRC vs. LOW	54.3 vs. 80.9	113.34 ($p < 0.001$)	0.138
NRC vs. NEUTRAL	54.3 vs. 48.8	4.41 ($p = 0.036$)	0.024

4.3 Register Effects & Prompt Sensitivity (Testing Hypothesis 2)

4.4 Lemma-Level Selectivity (Testing Hypothesis 3)

Table 7: Lemma-level idiomaticity across NRC and SCC conditions (% IDIOMATIC)

Lemma	NRC	HIGH	NEUTRAL	LOW
<i>break</i>	56.9	100.0	52.5	85.0
<i>drop</i>	40.6	95.5	17.4	81.4
<i>hold</i>	53.1	87.8	19.1	76.0
<i>keep</i>	56.6	34.7	28.9	59.2
<i>look</i>	48.5	90.7	42.9	90.0
<i>make</i>	58.3	90.0	58.3	84.0
<i>run</i>	35.0	100.0	81.0	78.0
<i>see</i>	33.6	91.8	74.0	82.1
<i>take</i>	75.7	95.9	58.3	92.0
<i>turn</i>	58.6	97.1	34.6	88.2

Note: SCC percentages are calculated from syntactically filtered cell sizes (see Section 3.3.1). NRC sample sizes vary reflecting natural frequency.

5 Discussion

5.1 Summary of Key Findings

5.2 Interpreting H1: The Pedagogical Simplification Hypothesis

5.3 Interpreting H2: Textbooks as Instructional Register

5.4 Interpreting H3: Lemma-Level Selectivity

5.5 The V-Shaped Pattern: Register Sensitivity in LLMs

5.6 Methodological Reflections

5.7 Limitations

5.8 Future Research

6 Conclusion

6.1 Research Summary

6.2 Theoretical Contributions

6.3 Practical Implications

6.4 Final Reflection

References

- Biber, Douglas (1988). *Variation across Speech and Writing*. 1st ed. Cambridge University Press. ISBN: 978-0-521-36543-7. DOI: 10.1017/CBO9780511621024.
- Biber, Douglas (1993). "Using Register-Diversified Corpora for General Language Studies". *Computational Linguistics* 19.2. Ed. by Julia Hirschberg, pp. 219–241. URL: <https://aclanthology.org/J93-2001/>.
- Biber, Douglas, Stig Johansson, Geoffrey Leech, Susan Conrad, and Edward Finegan (1999). *Longman Grammar of Spoken and Written English*. Edinburgh: Pearson Education Ltd.
- Conti, Gianfranco and Steve Smith (2016). *The ugly truth about school-based Modern Language teaching*. Critiques unnatural textbook approaches vs. natural language acquisition. URL: <https://gianfrancoconti.com/2016/11/26/the-ugly-truth-about-school-based-modern-language-teaching/>.
- Curdt-Christiansen, Xiao Lan and Csilla Weninger, eds. (2015). *Language, Ideology and Education: The Politics of Textbooks in Language Education*. (Routledge Research in Language Education). Hoboken: Taylor and Francis. ISBN: 978-1-317-80385-0 978-0-415-84038-5.
- Holmes, Wayne, Maya Bialik, and Charles Fadel (2019). *Artificial Intelligence in Education: Promises and Implications for Teaching and Learning*. Boston, MA: Center for Curriculum Redesign. ISBN: 978-1-7942-9370-0.
- Kia, Elnaz (2018). "Directive use in university classroom discourse: Variation across disciplines, academic levels, and interactivity". PhD thesis. Northern Arizona University.
- Krashen, Stephen D. (1985). *The Input Hypothesis: Issues and Implications*. London; New York: Longman. ISBN: 978-0-582-55381-1.
- Le Foll, Elen (2021a). *Bibliographic Metadata of the Textbook English Corpus (TEC). Textbook English: A Multi-Dimensional Approach*, v. 1.1. DOI: 10.5281/zenodo.4922819. (Visited on 01/31/2026).
- Le Foll, Elen (2021b). "Register variation in school EFL textbooks". *Register Studies* 3.2, pp. 207–246. DOI: 10.1075/rs.20009.lef. (Visited on 01/31/2026).
- Le Foll, Elen (2024). *Textbook English: A Multi-Dimensional Approach*. (Studies in Corpus Linguistics (SCL)) volume 116. Amsterdam Philadelphia: John Benjamins Publishing Company. ISBN: 978-90-272-4680-6. DOI: gruy-9789027246806.
- Quirk, Randolph, Sidney Greenbaum, Geoffrey Leech, and Jan Svartvik (1985). *A Comprehensive Grammar of the English Language*. London: Longman.
- Schmale, Günter (Feb. 2022). "Formulaic Expressions for Foreign Language Learning and Teaching". *Linguistik Online* 113.1, pp. 91–110. ISSN: 1615-3014. DOI: 10.13092/lo.113.8328. (Visited on 04/06/2026).
- Sinclair, John McHardy (1991). *Corpus, Concordance, Collocation*. (Describing English Language). Oxford: Oxford university press. ISBN: 978-0-19-437144-5.
- Spiro, Rand, Paul J. Feltovich, Michael Jacobson, and Richard Coulson (1992). "Cognitive Flexibility, Constructivism, and Hypertext: Random Access Instruction for Advanced Knowledge Acquisition in Ill-Structured Domains". *Constructivism and the technology of instruction: A conversation* 35, pp. 57–75.

Stoeckel, Tim (2019). "An Examination of the New General Service List". *Vocabulary Learning and Instruction* 8.1, pp. 53–61. DOI: 10.7820/vli.v08.1.stoeckel.

Sweller, John (1988). "Cognitive Load During Problem Solving: Effects on Learning". *Cognitive Science* 12.2, pp. 257–285. ISSN: 1551-6709. DOI: 10.1207/s15516709cog1202_4. (Visited on 01/31/2026).

Documentation of AI tools used

Table 8: Table documenting AI tools used

	AI Tool	Application	Used in section(s)	Comments
1	Gemini 2.5 Flash	Corpus Generation (SCC)	3.2.2	Via Python API
2	Gemini 2.5 Flash	Semantic Classification	3.3.4	LLM-as-a-judge
3	Gemini 3 Pro	Ideation & Structure	–	See screenshots
4	Claude Sonnet 4.5	Code & Debugging	Appendix D	–
5	Grammarly	Stylistic Proofreading	–	–

AI tools used

1. Gemini 2.5 Flash. Google. *Generate 100 sentences for the verb [lemma]* (Full prompt in Appendix D). 25/02/2026, <https://aistudio.google.com>.
2. Gemini 2.5 Flash. Google. *Classify the following sentence as LITERAL or ID-IOMATIC* (Full prompt in Appendix D). 28/02/2026, <https://aistudio.google.com>.
3. Gemini 3 Pro. Google. *I am starting my Bachelor's thesis in English Linguistics* (Full documentation in Appendix E). 20/11/2025, <https://gemini.google.com>.
4. Claude Sonnet 4.5. Anthropic. *Code & Debugging*. 21/11/2025-25/03/2026, <https://claude.ai>.
5. Grammarly. Grammarly. *Post-drafting spelling and grammatical review*. 28/03/2026, <https://grammarly.com>.

Plagiarism Statement

I hereby declare that I have independently written the present work and have not used any aids other than those indicated. Any passages taken verbatim or in substance from other works (including those from electronic databases or the internet) have been clearly marked as such, with proper citation according to academic referencing rules. This declaration also applies to any visual representations, tables, maps, and drawings included in the work. I know any deception will be penalised according to the applicable study and examination regulations.

A Large Language Model (LLM) was used during the ideation process; however, the text and ideas presented are my own unless otherwise specified.

A handwritten signature in black ink, consisting of stylized, cursive letters that appear to be 'S. J.' followed by a horizontal line.

Regensburg, April 16, 2026

A TEC Source Textbooks (B1/B2 Subset)

The following textbook titles (B1/B2-level subset) were included in the TEC-derived NRC sample:

- *Achievers B1* (2015)
- *Achievers B1+* (2015)
- *Achievers B2* (2015)
- *Hi there!* 3e, Niveau A2–B1 (2015)
- *Join the Team* 4e, A2–B1 (2012)
- *Join the Team* 3e, A2–B1 (2013)
- *New Missions* 2de, A2–B1 (2014)
- *Solutions Pre-Intermediate* (2017)
- *Solutions Intermediate* (2016)
- *Solutions Intermediate Plus* (2017)

B SCC Pilot Study Visualisations

The figures in this appendix document exploratory SCC pilot outputs used to refine prompt design and generation parameters before the full main study (see Section 3.2.3). They are included for transparency and should be read as preliminary diagnostics rather than final inferential results (see Figure 1 and Figure 2).

Shift in Semantic Usage by Register (Aggregate)

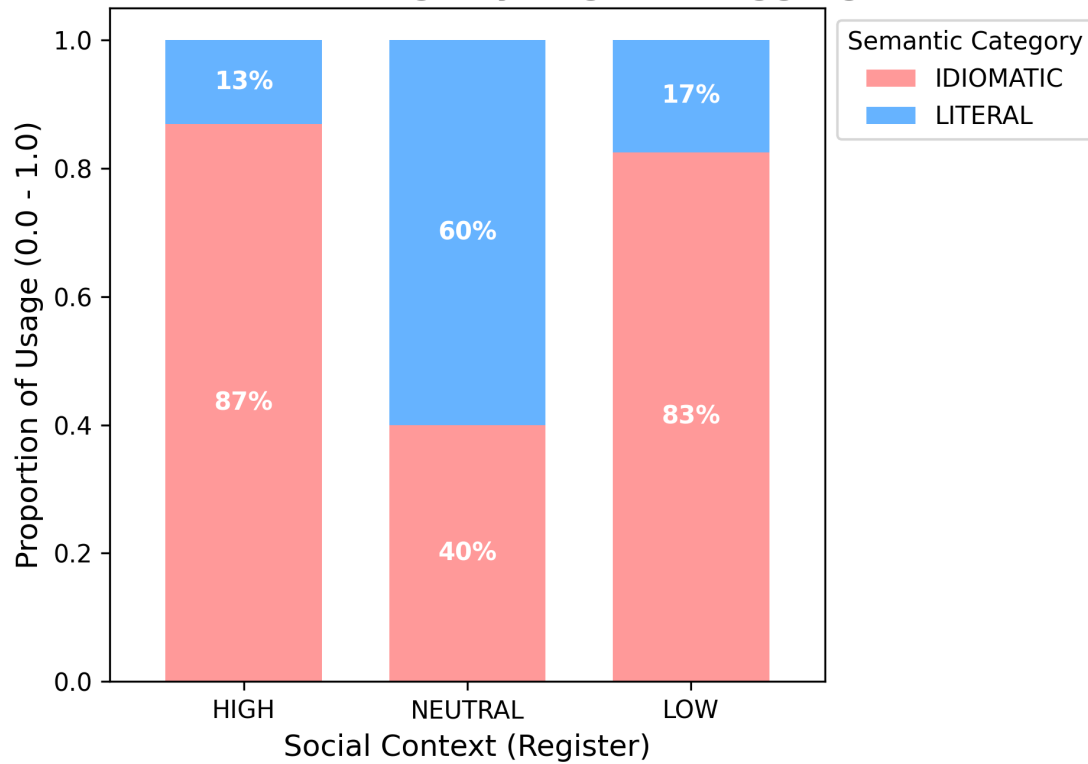


Figure 1: Aggregate proportion of `LITERAL` vs. `IDIOMATIC` usages by register (SCC pilot data).

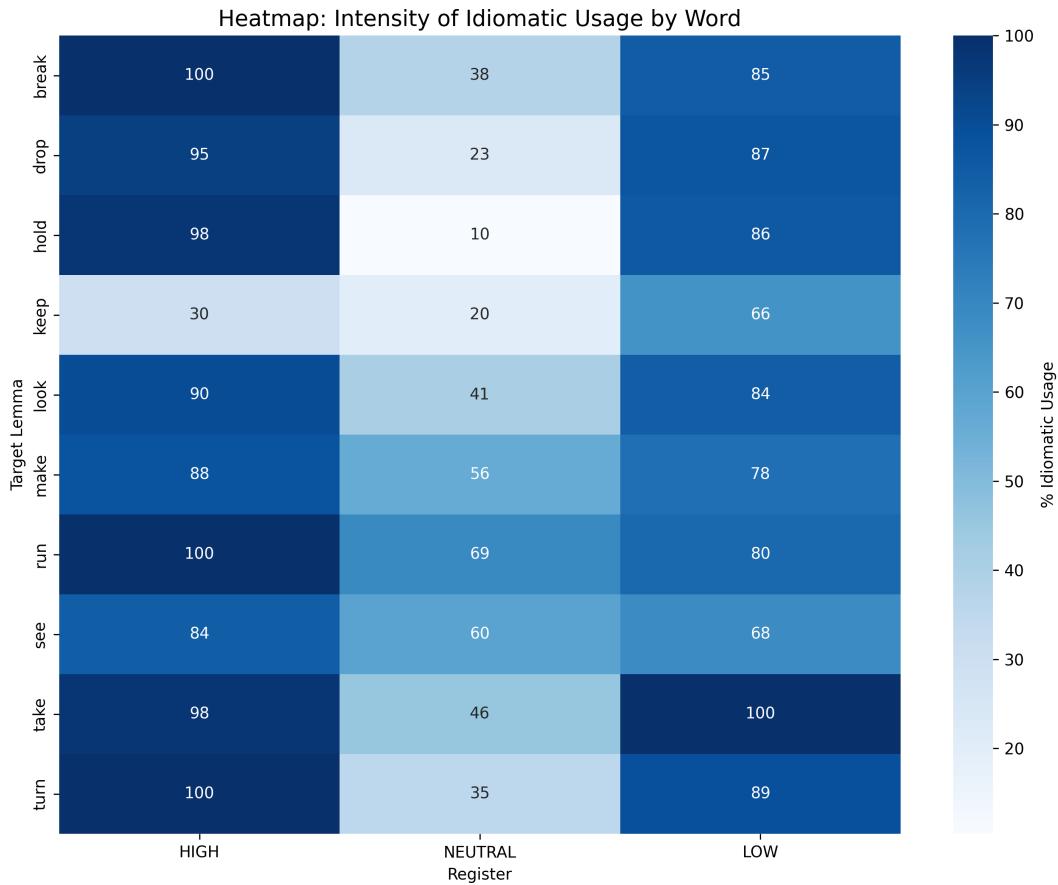


Figure 2: Per-lemma heatmap of `IDIOMATIC` usage rates by register (SCC pilot data).

B.1 Short Interpretation: *run*

In the pilot data, *run* is classified as 100% `IDIOMATIC` in the HIGH register. This aligns with the “Workplace/Professional” prompt condition: in professional contexts, *run* is typically used metaphorically (e.g. *run a meeting*, *run tests*, *run a project*) rather than to describe physical motion. For example, the synthetic HIGH-register sentence “Could you confirm if the new software will run efficiently on older systems?” uses *run* in the sense of “operate/function,” i.e. a non-literal (de-lexicalised) usage.

In the LOW register, *run* is 80% `IDIOMATIC`. This is expected because casual contexts plausibly include literal movement (e.g. *I ran to the store*), increasing the share of `LITERAL` uses.

B.2 Short Interpretation: *take*

The word *take* shows a V-shaped pattern. In HIGH (formal register), 98% of uses are `IDIOMATIC`, light-verb constructions like *take responsibility* or *take measures*, where *take* carries minimal semantic weight (e.g. from the SCC: “Might this project take an unexpected turn given the new market analysis?”). In NEUTRAL (instructions), this

drops to 46%, with *take* used literally (*Take an exit, Take one tablet*). In LOW (casual), idiomaticity rises back to 100%, because of phrasal verbs like *take off* or *take it easy* (e.g. from the SCC: “She always takes her time getting ready, it’s so annoying.”).

In the SCC pilot, both formal and casual registers show high idiomaticity, while the AI-generated instructional register shows substantially lower idiomaticity. This contrast motivated the subsequent NRC comparison, whose full inferential results are reported in Chapter 4.

C Final Dataset Visualisations

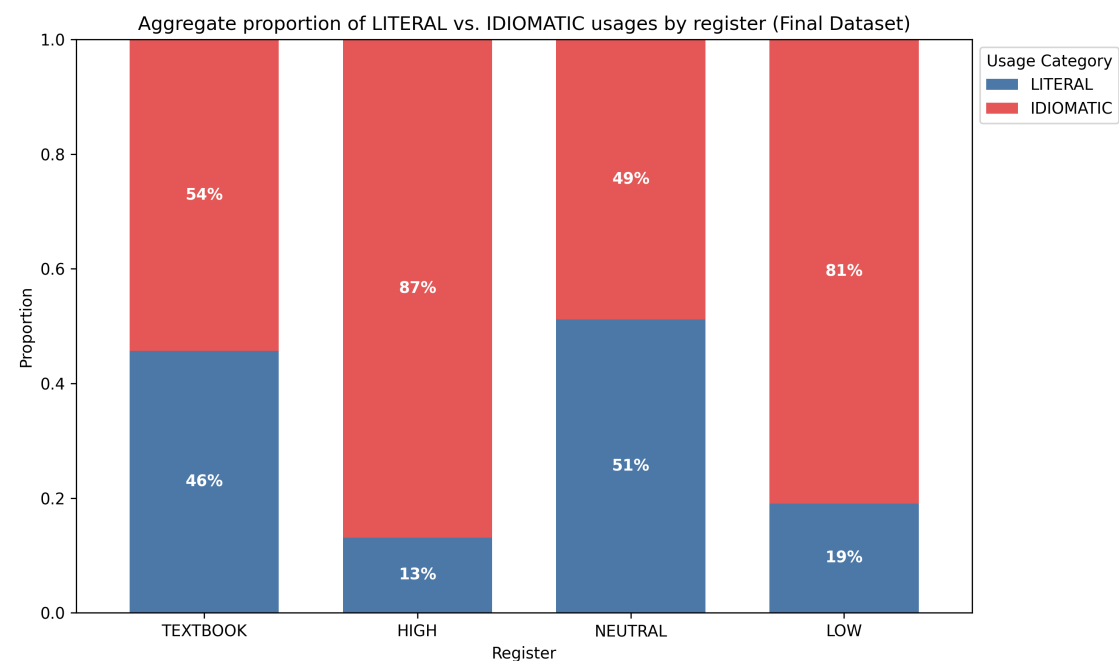


Figure 3: Aggregate proportion of LITERAL vs. IDIOMATIC usages by register (Final Dataset).

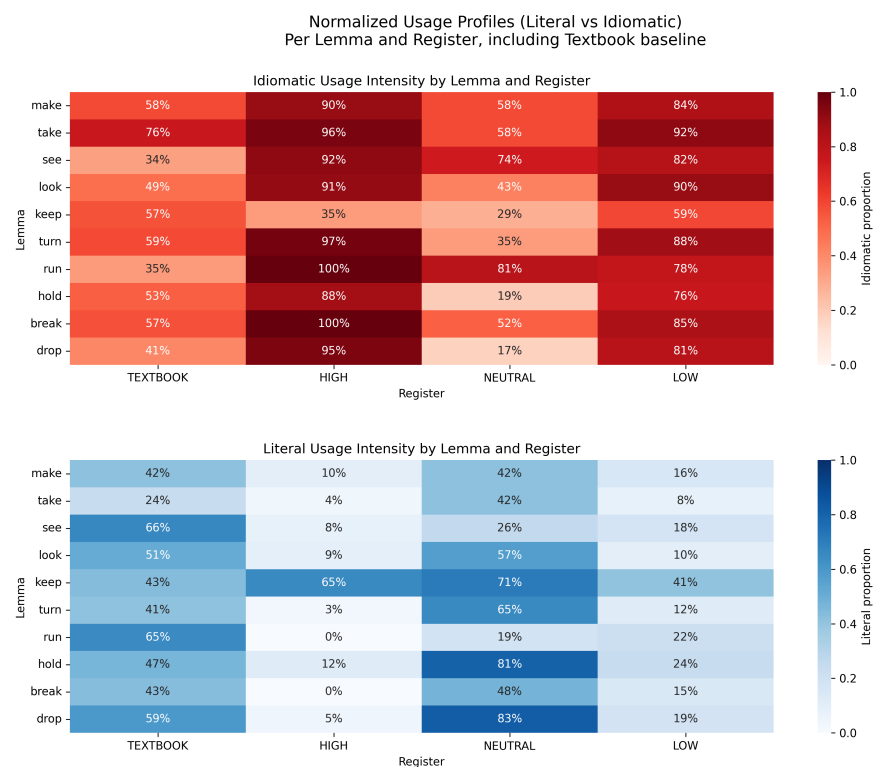


Figure 4: Per-lemma heatmap of IDIOMATIC usage rates by register (Final Dataset). Note: Cells displaying 100% idiomatcity (e.g., *run* in HIGH) are based on small cell sizes and should be interpreted with caution regarding absolute categorical boundaries.

These figures provide visual representations of the final semantic classification data ($N = 6,720$). Figure 3 shows the aggregate shift in idiomatic usage across prompt conditions, while Figure 4 shows the lemma-specific intensity of these phraseological shifts.

D Python Code Documentation

The following scripts document the computational pipeline. To ensure exact reproducibility, it must be noted that `pos_taggerV2NLTK.py` (Section D.5) was exclusively applied to filter the AI-generated Synthetic Control Corpus (SCC), while `pos_tagger_nltk_for_csv.py` (Section D.6) was engineered specifically to parse the human-authored Natural Reference Corpus (NRC). The code uses simplified file paths for readability. The actual repository implementation uses organised directories (`outputs/`, `data/`, `src/`) and environment variables for API keys. See the GitHub repository for the actual code, dataset and version history.

<https://github.com/Eidolom/llm-corpus-generation-analysis>

D.1 debug.py

```
import google.generativeai as genai

# Paste key directly here for the test
API_KEY = "API_KEY_HERE"

genai.configure(api_key=API_KEY)
model = genai.GenerativeModel('gemini-2.5-flash')

print("--- TESTING GEMINI API ---")

try:
    response = model.generate_content("Write a sentence using the word 'run'.")
    print(f"Success! Response: {response.text}")
except Exception as e:
    print(f"Error: {e}")
```

D.2 scalable_context_generator.py

```
import google.generativeai as genai
import time
import json
import re
from pathlib import Path

# --- CONFIGURATION ---
API_KEY = "API_KEY_HERE"
TARGET_WORDS_FILE = "data/target_words.txt"
```

```

OUTPUT_FILENAME = "outputs/intermediate_sentences.json"

# --- GENERATION SETTINGS ---
# 5 Batches x 10 sentences = 50 sentences per register (150 total per word)
NUM_BATCHES = 5
SENTENCES_PER_REG_PER_BATCH = 10

genai.configure(api_key=API_KEY)
model = genai.GenerativeModel('gemini-2.5-flash') # 2.5-flash is best for high-
    volume tasks

SYSTEM_PROMPT = """
You are a linguistic expert specialising in pragmatics and corpus generation.
Your task is to generate distinct, high-quality sentences for a target word
    based on strict register constraints.
The sentences must be appropriate for a CEFR B1/B2 learner.
"""

def clean_and_load_json(raw_text):
    """Clean markdown and parse JSON."""
    if not raw_text: return None
    try:
        # Regex to find the first [ and the last ]
        match = re.search(r'\[.*\]', raw_text, re.DOTALL)
        if match:
            return json.loads(match.group(0))
        else:
            # Fallback for simple strip
            cleaned = raw_text.replace('```json', '').replace('```', '').strip()
            ()
            return json.loads(cleaned)
    except Exception as e:
        print(f"    JSON Parse Error: {e}")
        return None

def load_target_words(filename):
    try:
        with open(filename, 'r', encoding='utf-8') as f:
            return [line.strip() for line in f if line.strip()]
    except FileNotFoundError:
        print(f"Error: '{filename}' not found.")
        return []

def generate_batch(word, batch_num):

```

```

"""Generates a single batch of 30 sentences (10 High, 10 Low, 10 Neutral)."""

prompt = f"""
Target Word: "{word}"
Batch ID: {batch_num} (Ensure these sentences are unique from generic
examples)

Generate a JSON array containing exactly {3 * SENTENCES_PER_REG_PER_BATCH}
sentences:

1. {SENTENCES_PER_REG_PER_BATCH} sentences: Register HIGH (Formal), Mood
Question. Context: Workplace/Professional.
2. {SENTENCES_PER_REG_PER_BATCH} sentences: Register LOW (Casual), Mood
Statement. Context: Friends/Personal/Slang.
3. {SENTENCES_PER_REG_PER_BATCH} sentences: Register NEUTRAL (Direct), Mood
Imperative. Context: Instructions/Navigation/Rules.

Vary the sentence structure and vocabulary usage.
Output JSON keys: "register", "mood", "sentence".
"""

try:
    response = model.generate_content(
        contents=[{"role": "user", "parts": [{"text": SYSTEM_PROMPT + "\n"
+ prompt}]]}
    )
    return clean_and_load_json(response.text)
except Exception as e:
    print(f"    API Error: {e}")
    return None

def main():
    words = load_target_words(TARGET_WORDS_FILE)
    if not words: return

    all_data = []
    total_words = len(words)

    print(f"--- STARTING GENERATION: {total_words} Words x {NUM_BATCHES}
    Batches ---")
    print(f"Target: {NUM_BATCHES * SENTENCES_PER_REG_PER_BATCH} sentences per
    register per word.")

```



```

for i, word in enumerate(words):
    word_sentences = []
    print(f"\n[{i+1}/{total_words}] Processing '{word}'...")

    for batch_idx in range(NUM_BATCHES):
        print(f"    -> Generating Batch {batch_idx+1}/{NUM_BATCHES}...", end
              = "", flush=True)

        batch_data = generate_batch(word, batch_idx)

        if batch_data:
            # Validate we actually got a list
            if isinstance(batch_data, list):
                # Add metadata
                for item in batch_data:
                    item['lemma'] = word
                    item['Source'] = "Synthetic_V2"
                    item['CEFR_Target'] = "B1/B2"
                    # Normalize register names just in case
                    if 'register' in item:
                        item['register'] = item['register'].upper()

                word_sentences.extend(batch_data)
                print(f"    Got {len(batch_data)} sentences.")
            else:
                print(f"    format error (not a list).")
        else:
            print("    Failed.")

        # Sleep slightly longer to avoid rate limits on the loop
        time.sleep(2.0)

    all_data.extend(word_sentences)
    print(f"    Total for '{word}': {len(word_sentences)} sentences.")

# Save
if all_data:
    Path(OUTPUT_FILENAME).parent.mkdir(parents=True, exist_ok=True)
    with open(OUTPUT_FILENAME, 'w', encoding='utf-8') as f:
        json.dump(all_data, f, indent=4)
    print(f"\nSUCCESS! Generated {len(all_data)} total sentences.")
    print(f"Saved to {OUTPUT_FILENAME}")
else:
    print("\n No data generated.")

```

```
if __name__ == "__main__":
    main()
```

D.3 target_words.txt

```
take
make
hold
keep
break
run
drop
turn
see
look
```

D.4 sketch_engine_converter_nltk.py

```
import json
import re
import pandas as pd
import nltk
from nltk.tokenize import sent_tokenize, word_tokenize
from nltk.tag import pos_tag

# --- NLTK SETUP ---
# NLTK requires downloading model files (tokenizers, taggers).
try:
    nltk.data.find('tokenizers/punkt_tab')
    nltk.data.find('taggers/averaged_perceptron_tagger_eng')
except LookupError:
    print("Downloading necessary NLTK models...")
    nltk.download('punkt_tab')
    nltk.download('averaged_perceptron_tagger_eng')

# --- CONFIGURATION ---
INPUT_FILE = 'textbook_sentences.json'
OUTPUT_JSON = 'semantic_analysis_results.json'
OUTPUT_CSV = 'semantic_analysis_summary.csv'

def clean_text(text):
    """Cleans messy textbook artifacts."""
```

```

if not text: return ""

# Remove double single quotes often found in SQL dumps
text = text.replace("'", "")

# Remove excessive whitespace
text = re.sub(r'\s+', ' ', text).strip()

return text


def get_context_window(words, index, window_size=5):
    """
    Since NLTK doesn't do full dependency parsing easily,
    we grab a 'context window' of words following the verb
    to capture the object/phrase (e.g., 'keep' -> 'in touch').
    """
    start = index + 1
    end = min(index + 1 + window_size, len(words))
    context_words = words[start:end]
    return " ".join(context_words)


def analyze_data(data):
    analyzed_rows = []
    print(f"Processing {len(data)} raw entries with NLTK...")

    for entry in data:
        raw_text = clean_text(entry.get('sentence', ''))
        target_lemma = entry.get('lemma', '').lower()

        # 1. Split chunk into sentences
        sentences = sent_tokenize(raw_text)

        for sent in sentences:
            # 2. Tokenize words
            words = word_tokenize(sent)

            # 3. Part-of-Speech Tagging
            # Returns list of tuples: [('Word', 'TAG'), ...]
            tagged_words = pos_tag(words)

            found_verb = False
            context_pattern = "N/A"

            for i, (word, tag) in enumerate(tagged_words):
                # Check if word matches target (normalized)
                # AND if tag starts with 'V' (VB, VBD, VBG, VBN, VBP, VBZ are
all verbs)

```

```

        if word.lower() == target_lemma and tag.startswith('V'):
            found_verb = True
            # Grab the words following the verb
            context_pattern = get_context_window(words, i)
            break

# 4. Filter and Save
# We filter out very short fragments (e.g. headers)
if found_verb and len(words) > 3:
    row = {
        "Target_Lemma": target_lemma,
        "Extracted_Sentence": sent,
        "Context_Pattern": f"{target_lemma} + {context_pattern}",
        "Original_Source": entry.get('Source', 'Unknown'),
        "Register": entry.get('register', 'N/A')
    }
    analyzed_rows.append(row)

return analyzed_rows

# --- MAIN EXECUTION ---
if __name__ == "__main__":
    print("--- STARTING SEMANTIC ANALYSIS (NLTK VERSION) ---")

    # 1. Load Data
    try:
        with open(INPUT_FILE, 'r', encoding='utf-8') as f:
            raw_data = json.load(f)
    except FileNotFoundError:
        print(f"Error: Could not find '{INPUT_FILE}'. Make sure it exists.")
        exit()

    # 2. Analyze
    results = analyze_data(raw_data)

    # 3. Save Results
    if results:
        with open(OUTPUT_JSON, 'w', encoding='utf-8') as f:
            json.dump(results, f, indent=4, ensure_ascii=False)

        df = pd.DataFrame(results)
        df.to_csv(OUTPUT_CSV, index=False)

    print(f"    Analysis Complete.")

```

```

        print(f"    Processed {len(raw_data)} chunks into {len(results)}
specific sentences.")

        print(f"    Saved to '{OUTPUT_CSV}'")

    else:

        print("No valid sentences containing the target verbs were found.")

```

D.5 pos_taggerV2NLTK.py

```

import sys
import json
import pandas as pd
from pathlib import Path
import nltk
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer

project_root = Path(__file__).parent.parent.parent
if str(project_root) not in sys.path:
    sys.path.insert(0, str(project_root))

INPUT_FILENAME = "outputs/intermediate_sentences.json"
OUTPUT_FILENAME = "outputs/thesis_pragmatic_data_filtered.csv"

nltk.download('punkt', quiet=True)
nltk.download('averaged_perceptron_tagger', quiet=True)
nltk.download('wordnet', quiet=True)

lemmatizer = WordNetLemmatizer()

def load_sentences(filename):
    """Loads sentences from the intermediate JSON file."""
    try:
        with open(filename, 'r', encoding='utf-8') as f:
            return json.load(f)
    except Exception as e:
        print(f"Error loading file: {e}")
        return None

def main():
    print("--- STARTING NLTK POS-GATEKEEPER ---")
    all_sentences = load_sentences(INPUT_FILENAME)
    if not all_sentences:
        return

```

```

print(f"Total raw sentences loaded: {len(all_sentences)}")

valid_sentences = []
dropped_count = 0

for item in all_sentences:
    target_lemma = item['lemma'].lower()
    sentence_text = item['sentence']

    tokens = word_tokenize(sentence_text)
    pos_tags = nltk.pos_tag(tokens)

    is_valid_verb = False

    for token, tag in pos_tags:
        if tag.startswith('V'):
            token_lemma = lemmatizer.lemmatize(token.lower(), pos='v')

            if token_lemma == target_lemma:
                is_valid_verb = True
                break

    if is_valid_verb:
        valid_sentences.append(item)
    else:
        dropped_count += 1

print(f"\nFiltering Complete!")
print(f"Sentences Kept: {len(valid_sentences)}")
print(f"Sentences Dropped (Nouns/Adjectives): {dropped_count}")

if valid_sentences:
    df = pd.DataFrame(valid_sentences)
    df.to_csv(OUTPUT_FILENAME, index=False)
    print(f"Clean, verb-only dataset saved to '{OUTPUT_FILENAME}'")

else:
    print(f"\nFailed to generate the final CSV. No valid verbs found.")

if __name__ == "__main__":
    main()

```

D.6 pos_tagger_nltk_for_csv.py

```

import pandas as pd
import nltk
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer

# --- CONFIGURATION ---
INPUT_FILENAME = "outputs/thesis_semantic_data_final.csv" # Update path if
    needed
OUTPUT_FILENAME = "outputs/thesis_textbook_data_filtered.csv" # Update
    path if needed

# Download required NLTK datasets (quietly)
nltk.download('punkt', quiet=True)
nltk.download('averaged_perceptron_tagger', quiet=True)
nltk.download('wordnet', quiet=True)

lemmatizer = WordNetLemmatizer()

def main():
    print("--- STARTING NLTK POS-GATEKEEPER (CSV VERSION) ---")

    try:
        # Load the CSV
        df = pd.read_csv(INPUT_FILENAME)
    except Exception as e:
        print(f"Error loading CSV: {e}")
        return

    print(f"Total raw sentences loaded: {len(df)}")

    valid_indices = []

    # Identify the correct column names (handling uppercase/lowercase
    differences)
    text_col = 'Full_Sentence' if 'Full_Sentence' in df.columns else 'sentence'
    lemma_col = 'Lemma' if 'Lemma' in df.columns else 'lemma'

    # 1. The Lemmatize-Before-Filtering Logic
    for index, row in df.iterrows():
        target_lemma = str(row[lemma_col]).lower().strip()
        sentence_text = str(row[text_col])

        # Tokenize and tag the sentence

```

```

tokens = word_tokenize(sentence_text)
pos_tags = nltk.pos_tag(tokens)

is_valid_verb = False

# Scan the sentence for our target verb
for token, tag in pos_tags:
    # Check if NLTK tagged it as ANY type of verb (VB, VBD, VBG, VBN, VBP, VBZ)
    if tag.startswith('V'):
        # Lemmatise the verb
        token_lemma = lemmatizer.lemmatize(token.lower(), pos='v')

        # Check if it matches our target lemma
        if token_lemma == target_lemma:
            is_valid_verb = True
            break # Found it acting as a verb, keep the row

if is_valid_verb:
    valid_indices.append(index)

# 2. Filter the dataframe to only keep valid rows
clean_df = df.loc[valid_indices]
dropped_count = len(df) - len(clean_df)

print(f"\nFiltering Complete!")
print(f"Sentences Kept: {len(clean_df)}")
print(f"Sentences Dropped (Nouns/Adjectives): {dropped_count}")

# 3. Export the clean data
if not clean_df.empty:
    from pathlib import Path
    Path(OUTPUT_FILENAME).parent.mkdir(parents=True, exist_ok=True)
    clean_df.to_csv(OUTPUT_FILENAME, index=False)
    print(f"Clean, verb-only dataset saved to '{OUTPUT_FILENAME}'")
else:
    print(f"\nFailed to generate the final CSV. No valid verbs found.")

if __name__ == "__main__":
    main()

```

D.7 semantic_analyzer.py

```

import google.generativeai as genai

```



```

import pandas as pd
import time
import json
import re
import math
from pathlib import Path

# --- CONFIGURATION ---
API_KEY = "API_KEY_HERE"
INPUT_FILENAME = "outputs/intermediate_sentences.json"
OUTPUT_FILENAME = "outputs/thesis_semantic_data_final.csv"

# BATCH SIZE FOR ANALYSIS
# We will process 20 sentences at a time. This is the "Safe Zone" for LLM
# counting.
CHUNK_SIZE = 20

genai.configure(api_key=API_KEY)
model = genai.GenerativeModel('gemini-2.5-flash')

SYSTEM_PROMPT = """
You are a semantic analyzer. I will provide a list of sentences containing a
target word.
Classify the usage of the target word in each sentence into exactly one
category:
1. "LITERAL" (Physical/Core meaning).
2. "IDIOMATIC" (De-lexicalized, Metaphor, Phrasal Verb, or Fixed Phrase).

Return a strict JSON ARRAY of strings.
Example Output: ["LITERAL", "IDIOMATIC", "LITERAL"]
"""

def load_sentences(filename):
    try:
        with open(filename, 'r', encoding='utf-8') as f:
            return json.load(f)
    except Exception as e:
        print(f" Error loading input file: {e}")
        return None

def get_tags_for_chunk(word, chunk_sentences, retry_count=0):
    """
    Analyzes a small chunk of sentences.
    """

```

```

prompt = f"""
Target Word: "{word}"
Sentences to classify ({len(chunk_sentences)} items):
{json.dumps(chunk_sentences)}

Return exactly {len(chunk_sentences)} tags in a JSON array.
"""

try:
    response = model.generate_content(
        contents=[{"role": "user", "parts": [{"text": SYSTEM_PROMPT + "\n"
+ prompt}]]]
    )

    raw_text = response.text.strip()

    # Cleaner Regex to find JSON array
    match = re.search(r'\[.*?\]', raw_text, re.DOTALL)
    if not match:
        # If regex fails, try parsing raw text if it looks like a list
        if raw_text.startswith('[') and raw_text.endswith(']'):
            json_str = raw_text
        else:
            raise ValueError("No JSON array found")
    else:
        json_str = match.group(0)

    tags = json.loads(json_str)

    # Validate Count
    if len(tags) != len(chunk_sentences):
        # Retry once if count is wrong
        if retry_count < 1:
            print(f"    Count mismatch ({len(tags)} vs {len(
chunk_sentences)}). Retrying...")
            time.sleep(1)
            return get_tags_for_chunk(word, chunk_sentences, retry_count=1)
        else:
            print(f"    Failed chunk for '{word}'. Expected {len(
chunk_sentences)}, got {len(tags)}.")
            return ["ERROR"] * len(chunk_sentences)

return [str(t).upper() for t in tags]

```

```

except Exception as e:
    print(f"      Error: {e}")
    return ["ERROR"] * len(chunk_sentences)

def main():
    data = load_sentences(INPUT_FILENAME)
    if not data: return

    # Group by Lemma
    grouped_data = {}
    for item in data:
        lemma = item['lemma']
        if lemma not in grouped_data: grouped_data[lemma] = []
        grouped_data[lemma].append(item)

    final_rows = []
    sorted_words = sorted(grouped_data.keys())
    total_words = len(sorted_words)

    print(f"--- STARTING CHUNKED ANALYSIS FOR {total_words} WORDS ---")

    for i, word in enumerate(sorted_words):
        records = grouped_data[word]
        total_items = len(records)
        print(f"[{i+1}/{total_words}] Analyzing '{word}' ({total_items}
sentences)...")

        # --- THE CHUNKING LOOP ---
        word_tags = []
        num_chunks = math.ceil(total_items / CHUNK_SIZE)

        for batch_idx in range(num_chunks):
            start = batch_idx * CHUNK_SIZE
            end = start + CHUNK_SIZE
            chunk_records = records[start:end]
            chunk_sentences = [r['sentence'] for r in chunk_records]

            # Get tags for just this small batch
            tags = get_tags_for_chunk(word, chunk_sentences)
            word_tags.extend(tags)

            # print(f"      Batch {batch_idx+1}/{num_chunks}: Processed {len(tags)
}) items.")

            time.sleep(0.5) # Fast sleep for small chunks

```

```

# Merge tags back
for record, tag in zip(records, word_tags):
    row = {
        "Lemma": record['lemma'],
        "Register": record['register'],
        "Mood": record['mood'],
        "Usage_Category": tag,
        "Full_Sentence": record['sentence']
    }
    final_rows.append(row)

# Save
df = pd.DataFrame(final_rows)
Path(OUTPUT_FILENAME).parent.mkdir(parents=True, exist_ok=True)
df.to_csv(OUTPUT_FILENAME, index=False)

print("\n" + "="*40)
print(f" DONE. Saved to {OUTPUT_FILENAME}")
print("Summary of Results:")
print(df['Usage_Category'].value_counts())
print("="*40)

if __name__ == "__main__":
    main()

```

D.8 irr_sheet_generator.py

```

"""
Generate blinded IRR annotation material for the thesis workflow.

Outputs:
1) Files/irr_annotation_sheet.csv
   - Sheet used for manual coding (no model labels or register metadata).
2) Files/irr_gold_key.csv
   - Hidden key containing model labels for later agreement calculation.

Design:
- 100 NRC items (TEXTBOOK)
- 100 SCC items (stratified across HIGH / NEUTRAL / LOW)
- Deterministic random seed for reproducibility
"""

import pandas as pd

```

```

# --- CONFIGURATION ---
NRC_INPUT_FILE = "Files/thesis_semantic_data_final.csv"
SCC_INPUT_FILE = "Files/thesis_semantic_data_final_2.csv"

ANNOTATION_OUTPUT_FILE = "Files/irr_annotation_sheet.csv"
KEY_OUTPUT_FILE = "Files/irr_gold_key.csv"

NRC_SAMPLE_SIZE = 100
SCC_SAMPLE_SIZE = 100
RANDOM_SEED = 42

REQUIRED_COLUMNS = ["Lemma", "Register", "Mood", "Usage_Category", "
    Full_Sentence"]
SCC_REGISTER_ORDER = ["HIGH", "NEUTRAL", "LOW"]

def validate_columns(df: pd.DataFrame, source_name: str) -> None:
    missing = [column for column in REQUIRED_COLUMNS if column not in df.
        columns]
    if missing:
        raise ValueError(f"{source_name} is missing required columns: {missing}
            ")

def sample_nrc(nrc_df: pd.DataFrame, n: int, seed: int) -> pd.DataFrame:
    nrc_pool = nrc_df[nrc_df["Register"].astype(str).str.upper() == "TEXTBOOK"
        ].copy()
    if len(nrc_pool) < n:
        raise ValueError(f"NRC pool too small: requested {n}, available {len(
            nrc_pool)}")

    sample = nrc_pool.sample(n=n, random_state=seed).copy()
    sample["Source_Group"] = "NRC"
    return sample

def sample_scc_stratified(scc_df: pd.DataFrame, n_total: int, seed: int) -> pd.
    DataFrame:
    scc_df = scc_df.copy()
    scc_df["Register"] = scc_df["Register"].astype(str).str.upper()

```

```

missing_registers = [reg for reg in SCC_REGISTER_ORDER if reg not in set(
scc_df["Register"])]
if missing_registers:
    raise ValueError(f"SCC data is missing required registers: {
missing_registers}")

base = n_total // len(SCC_REGISTER_ORDER)
remainder = n_total % len(SCC_REGISTER_ORDER)

allocations = {
    register: base + (1 if idx < remainder else 0)
    for idx, register in enumerate(SCC_REGISTER_ORDER)
}

sampled_chunks = []
for idx, register in enumerate(SCC_REGISTER_ORDER):
    n_register = allocations[register]
    register_pool = scc_df[scc_df["Register"] == register]

    if len(register_pool) < n_register:
        raise ValueError(
            f"SCC pool for {register} too small: requested {n_register},
available {len(register_pool)}"
        )

    sampled_chunk = register_pool.sample(n=n_register, random_state=seed +
idx).copy()
    sampled_chunks.append(sampled_chunk)

sample = pd.concat(sampled_chunks, ignore_index=True)
sample["Source_Group"] = "SCC"
return sample

def build_outputs(nrc_sample: pd.DataFrame, scc_sample: pd.DataFrame, seed: int
) -> tuple[pd.DataFrame, pd.DataFrame]:
    combined = pd.concat([nrc_sample, scc_sample], ignore_index=True)
    combined = combined.sample(frac=1.0, random_state=seed).reset_index(drop=
True)

    combined["IRR_ID"] = [f"IRR_{idx:04d}" for idx in range(1, len(combined) +
1)]
    combined["Model_Label"] = combined["Usage_Category"].astype(str).str.upper
()

```

```

annotation_sheet = combined[["IRR_ID", "Lemma", "Full_Sentence"]].rename(
    columns={
        "IRR_ID": "irr_id",
        "Lemma": "lemma",
        "Full_Sentence": "sentence",
    }
)
annotation_sheet["human_label"] = ""
annotation_sheet["notes"] = ""

gold_key = combined[
    [
        "IRR_ID",
        "Source_Group",
        "Register",
        "Mood",
        "Lemma",
        "Model_Label",
        "Full_Sentence",
    ]
].rename(
    columns={
        "IRR_ID": "irr_id",
        "Source_Group": "source_group",
        "Register": "register",
        "Mood": "mood",
        "Lemma": "lemma",
        "Model_Label": "model_label",
        "Full_Sentence": "sentence",
    }
)

return annotation_sheet, gold_key

```

```

def main() -> None:
    print("--- Generating IRR annotation sheet ---")

    nrc_df = pd.read_csv(NRC_INPUT_FILE)
    scc_df = pd.read_csv(SCC_INPUT_FILE)

    validate_columns(nrc_df, "NRC input")
    validate_columns(scc_df, "SCC input")

```

```

nrc_sample = sample_nrc(nrc_df, n=NRC_SAMPLE_SIZE, seed=RANDOM_SEED)
scc_sample = sample_scc_stratified(scc_df, n_total=SCC_SAMPLE_SIZE, seed=
RANDOM_SEED)

annotation_sheet, gold_key = build_outputs(nrc_sample, scc_sample, seed=
RANDOM_SEED)

annotation_sheet.to_csv(ANNOTATION_OUTPUT_FILE, index=False)
gold_key.to_csv(KEY_OUTPUT_FILE, index=False)

print(f"Saved annotation sheet: {ANNOTATION_OUTPUT_FILE}")
print(f"Saved gold key: {KEY_OUTPUT_FILE}")
print(f"Total IRR sample size: {len(annotation_sheet)}")

source_counts = gold_key["source_group"].value_counts().to_dict()
register_counts = gold_key["register"].value_counts().to_dict()
print(f"Source balance: {source_counts}")
print(f"Register balance: {register_counts}")

if __name__ == "__main__":
    main()

```

D.9 compute_split_irr.py

```

"""
Split IRR analysis for NRC vs SCC.

This script compares human labels to model labels and reports:
- aggregate Cohen's kappa
- NRC-only Cohen's kappa
- SCC-only Cohen's kappa

Expected inputs
-----
- Files/irr_annotation_sheet - irr_annotation_sheet.csv.csv
  (human labels)
- Files/irr_master_key.csv (optional)
  (model labels + metadata)

If a master-key file is unavailable, the script can read a merged sheet
that already contains both human and model labels.
"""

```



```

from __future__ import annotations

from pathlib import Path
from dataclasses import dataclass
import re

import pandas as pd
from sklearn.metrics import cohen_kappa_score

VALID_LABELS = {"LITERAL", "IDIOMATIC"}

@dataclass
class SplitResult:
    group: str
    n_valid: int
    raw_agreement: float | None
    kappa: float | None

def normalize_text(value: object) -> str:
    text = "" if value is None else str(value)
    text = re.sub(r"\s+", " ", text)
    return text.strip().lower()

def normalize_label(value: object) -> str:
    label = "" if value is None else str(value)
    return label.strip().upper()

def first_existing(columns: list[str], candidates: list[str]) -> str | None:
    lower_map = {c.lower(): c for c in columns}
    for candidate in candidates:
        key = candidate.lower()
        if key in lower_map:
            return lower_map[key]
    return None

def compute_group(df: pd.DataFrame, group_name: str) -> SplitResult:
    clean = df.copy()

```

```

clean["human_label"] = clean["human_label"].map(normalize_label)
clean["model_label"] = clean["model_label"].map(normalize_label)

keep = (
    clean["human_label"].isin(VALID_LABELS)
    & clean["model_label"].isin(VALID_LABELS)
)
clean = clean[keep]

if clean.empty:
    return SplitResult(group_name, 0, None, None)

raw = (clean["human_label"] == clean["model_label"]).mean()
kappa = cohen_kappa_score(clean["human_label"], clean["model_label"])
return SplitResult(group_name, int(len(clean)), float(raw), float(kappa))

def kappa_interpretation(value: float | None) -> str:
    if value is None:
        return "n/a"
    if value < 0.00:
        return "poor"
    if value < 0.20:
        return "slight"
    if value < 0.40:
        return "fair"
    if value < 0.60:
        return "moderate"
    if value < 0.80:
        return "substantial"
    return "almost perfect"

def load_input_frames(project_root: Path) -> pd.DataFrame:
    files_dir = project_root / "Files"

    annotation_path = (
        files_dir / "irr_annotation_sheet - irr_annotation_sheet.csv.csv"
    )
    master_key_path = files_dir / "irr_master_key.csv"

    if not annotation_path.exists():
        raise FileNotFoundError(
            f"Missing annotation file: {annotation_path}"
        )

```

```

    )

ann = pd.read_csv(annotation_path)

human_col = first_existing(
    ann.columns.tolist(),
    ["Human_Label", "human_label", "human", "label_human"],
)

sent_col = first_existing(
    ann.columns.tolist(),
    ["Full_Sentence", "full_sentence", "sentence", "text"],
)

if human_col is None:
    raise ValueError("Could not find human label column.")

ann = ann.rename(columns={human_col: "human_label"})

if master_key_path.exists() and sent_col is not None:
    key = pd.read_csv(master_key_path)
    key_sent = first_existing(
        key.columns.tolist(),
        ["Full_Sentence", "full_sentence", "sentence", "text"],
    )
    key_model = first_existing(
        key.columns.tolist(),
        ["model_label", "Model_Label", "ai_label", "label_model"],
    )
    key_group = first_existing(
        key.columns.tolist(),
        ["source_group", "Source_Group", "group", "corpus"],
    )

    if key_sent and key_model and key_group and sent_col:
        ann["_sent_key"] = ann[sent_col].map(normalize_text)
        key["_sent_key"] = key[key_sent].map(normalize_text)
        keep_cols = ["_sent_key", key_model, key_group]
        key = key[keep_cols].drop_duplicates("_sent_key")
        key = key.rename(
            columns={
                key_model: "model_label",
                key_group: "source_group",
            }
        )

```

```

        merged = ann.merge(key, on="_sent_key", how="left")
        return merged

model_col = first_existing(
    ann.columns.tolist(),
    ["Model_Label", "model_label", "ai_label", "label_model"],
)
group_col = first_existing(
    ann.columns.tolist(),
    ["Source_Group", "source_group", "group", "corpus"],
)

if model_col is None or group_col is None:
    raise ValueError(
        "Need either Files/irr_master_key.csv or columns "
        "for model labels and source group in annotation sheet."
    )

ann = ann.rename(columns={model_col: "model_label", group_col: "
source_group"})
return ann

def main() -> None:
    project_root = Path(__file__).resolve().parents[1]
    df = load_input_frames(project_root)

    if "source_group" not in df.columns:
        raise ValueError("Missing source_group column.")
    if "model_label" not in df.columns:
        raise ValueError("Missing model_label column.")

    df["source_group"] = df["source_group"].astype(str).str.upper().str.strip()

    aggregate = compute_group(df, "ALL")
    nrc = compute_group(df[df["source_group"] == "NRC"], "NRC")
    scc = compute_group(df[df["source_group"] == "SCC"], "SCC")

    rows = [aggregate, nrc, scc]

    print("=" * 64)
    print(f"{'Group':<10}{'n':>8}{'Raw Agr.':>12}{'Kappa':>10} Interpretation"
    )
    print("-" * 64)

```

```

for r in rows:
    if r.kappa is None:
        print(f"{r.group:<10}{'-':>8}{'-':>12}{'-':>10}  n/a")
        continue
    raw_pct = f"{r.raw_agreement * 100:.1f}%"
    print(
        f"{r.group:<10}{r.n_valid:>8}{raw_pct:>12}"
        f"{r.kappa:>10.3f}  {kappa_interpretation(r.kappa)}"
    )
print("=" * 64)

out_dir = project_root / "outputs"
out_dir.mkdir(exist_ok=True)
out_path = out_dir / "irr_split_results.csv"

out_df = pd.DataFrame(
    {
        "group": [r.group for r in rows],
        "n_valid": [r.n_valid for r in rows],
        "raw_agreement": [r.raw_agreement for r in rows],
        "kappa": [r.kappa for r in rows],
        "interpretation": [kappa_interpretation(r.kappa) for r in rows],
    }
)
out_df.to_csv(out_path, index=False)
print(f"Saved: {out_path}")

if __name__ == "__main__":
    main()

```

D.10 irr_split_results.csv

```

group,n_valid,n_excluded,raw_agreement,kappa
ALL (aggregate),186,14,0.8118,0.6158
NRC (Textbook),87,13,0.7701,0.532
SCC (AI-generated),99,1,0.8485,0.6379

```

D.11 visualizer.py

```

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

```

```

from itertools import combinations
from itertools import combinations
from scipy.stats import chi2_contingency, fisher_exact, fisher_exact

# --- CONFIGURATION ---
INPUT_FILENAME = "thesis_semantic_data_final.csv"

# 1. Load Data
df = pd.read_csv(INPUT_FILENAME)

# Clean up Register names just in case (ensure UPPERCASE)
df['Register'] = df['Register'].str.upper()

# Define specific order for charts (Logical progression)
register_order = ['HIGH', 'NEUTRAL', 'LOW']
usage_order = ['LITERAL', 'IDIOMATIC']

# --- PART 1: THE MAIN FINDING (Stacked Bar Chart) ---
print("\n--- 1. GENERATING AGGREGATE CHART ---")

# Calculate counts and percentages
cross_tab = pd.crosstab(df['Register'], df['Usage_Category'])
cross_tab = cross_tab.reindex(index=register_order, columns=usage_order,
                              fill_value=0)
cross_tab = cross_tab.loc[cross_tab.sum(axis=1) > 0]
cross_tab_prop = cross_tab.div(cross_tab.sum(1), axis=0)

# Plotting
plt.figure(figsize=(10, 6))
# Plot the bars (stacked)
ax = cross_tab_prop.plot(kind='bar', stacked=True, color=['#ff9999', '#66b3ff'],
                        width=0.7)

# Formatting
plt.title('Shift in Semantic Usage by Register (Aggregate)', fontsize=14,
         fontweight='bold')
plt.ylabel('Proportion of Usage (0.0 - 1.0)', fontsize=12)
plt.xlabel('Social Context (Register)', fontsize=12)
plt.xticks(rotation=0)
plt.legend(title='Semantic Category', loc='upper left', bbox_to_anchor=(1, 1))

# Add percentage labels on the bars
for c in ax.containers:

```

```

    labels = [f"{value * 100:.0f}%" if value > 0 else "" for value in c.
    datavalues]

    ax.bar_label(c, labels=labels, label_type='center', color='white',
    fontweight='bold')

plt.tight_layout()
plt.savefig('results_aggregate_chart.png', dpi=300)
print(" Saved 'results_aggregate_chart.png'")

# --- PART 2: THE STATISTICAL TEST ---
print("\n--- 2. RUNNING CHI-SQUARE TEST ---")
# We test if Register and Usage are independent
chi2, p, dof, expected = chi2_contingency(cross_tab)

print(f"Chi-Square Statistic: {chi2:.4f}")
print(f"P-Value: {p:.4e}")
if p < 0.05:
    print(">> RESULT: Statistically Significant! (You can reject the Null
    Hypothesis)")
else:
    print(">> RESULT: Not Significant.")

# Follow-up analysis for sparse expected frequencies
if (expected < 5).any():
    print("\nExpected frequencies below 5 detected. Running pairwise 2x2 Fisher
    follow-up tests.")
    pairwise_results = []
    for reg_a, reg_b in combinations(cross_tab.index, 2):
        pair_table = cross_tab.loc[[reg_a, reg_b], usage_order]
        if (pair_table.sum(axis=1) == 0).any() or (pair_table.sum(axis=0) == 0)
        .any():
            continue
        odds_ratio, fisher_p = fisher_exact(pair_table.values)
        pairwise_results.append((reg_a, reg_b, odds_ratio, fisher_p))

    if pairwise_results:
        corrected_alpha = 0.05 / len(pairwise_results)
        print(f"Bonferroni-corrected alpha: {corrected_alpha:.4f}")
        for reg_a, reg_b, odds_ratio, fisher_p in pairwise_results:
            significance = "significant" if fisher_p < corrected_alpha else "
            not significant"

            print(
                f"Fisher {reg_a} vs {reg_b}: p={fisher_p:.4e}, "
                f"odds_ratio={odds_ratio:.4f} ({significance})"

```

```

        )

    else:
        print("Pairwise Fisher follow-up skipped (insufficient variation in 2x2
        tables).")

# --- PART 3: THE WORD BREAKDOWN (Heatmap) ---
print("\n--- 3. GENERATING WORD HEATMAP ---")

# Calculate % Idiomatic for every word in every register
pivot = df.groupby(['Lemma', 'Register'])['Usage_Category'].value_counts(
    normalize=True).unstack(fill_value=0)

# We only care about the "IDIOMATIC" column for the heatmap
if 'IDIOMATIC' in pivot.columns:
    heatmap_data = pivot['IDIOMATIC'].unstack().reindex(columns=register_order)

    plt.figure(figsize=(10, 8))
    sns.heatmap(heatmap_data, annot=True, cmap="Blues", fmt=".0%", cbar_kws={'
    label': '% Idiomatic Usage'})
    plt.title('Heatmap: Intensity of Idiomatic Usage by Word', fontsize=14)
    plt.ylabel('Target Lemma')
    plt.xlabel('Register')
    plt.tight_layout()
    plt.savefig('results_heatmap.png', dpi=300)
    print("Saved 'results_heatmap.png'")
else:
    print("Could not generate heatmap (Label 'IDIOMATIC' missing?)")

print("\nDONE. Open the PNG files to see your thesis results.")

```


E Full AI Documentation

E.1 Gemini 2.5 Flash: SCC Generation Prompts

Date used: 05/02/2026

System Prompt

You are a linguistic expert specialising in pragmatics and corpus generation. Your task is to generate distinct, high-quality sentences for a target word based on strict register constraints. The sentences must be appropriate for a CEFR B1/B2 learner.

Batch Prompt

Target Word: "{word}"
Batch ID: {batch_num} (Ensure these sentences are unique from generic examples)

Generate a JSON array containing exactly {3 * SENTENCES_PER_REG_PER_BATCH} sentences:

- {SENTENCES_PER_REG_PER_BATCH} sentences: Register HIGH (Formal), Mood Question. Context: Workplace/Professional.
- {SENTENCES_PER_REG_PER_BATCH} sentences: Register LOW (Casual), Mood Statement. Context: Friends/Personal/Slang.
- {SENTENCES_PER_REG_PER_BATCH} sentences: Register NEUTRAL (Direct), Mood Imperative. Context: Instructions/Navigation/Rules.

Vary the sentence structure and vocabulary usage.
Output JSON keys: "register", "mood", "sentence".

E.2 Gemini 2.5 Flash: Semantic Classification Prompts

Date used: 28/02/2026

System Prompt

You are a semantic analyzer. I will provide a list of sentences containing a target word. Classify the usage of the target word in each sentence into exactly one category:

- "LITERAL" (Physical/Core meaning).
- "IDIOMATIC" (De-lexicalized, Metaphor, Phrasal Verb, or Fixed Phrase).

Return a strict JSON ARRAY of strings.
Example Output: ["LITERAL", "IDIOMATIC", "LITERAL"]

Chunk Prompt

```
Target Word: "{word}"
Sentences to classify ({len(chunk_sentences)} items):
{json.dumps(chunk_sentences)}

Return exactly {len(chunk_sentences)} tags in a JSON array.
```

E.3 Gemini 3 Pro: Ideation Prompt

Date used: 20/11/2025

I am starting my Bachelor's thesis in English Linguistics. I want to compare "Textbook English" idiomaticity against sentences generated by an LLM like Gemini. I am thinking of building two corpora and using a "High/Neutral/Low" register distinction for the AI side to test whether it can mimic pedagogical styles. Can you help me brainstorm a formal academic structure for this?

Screenshots

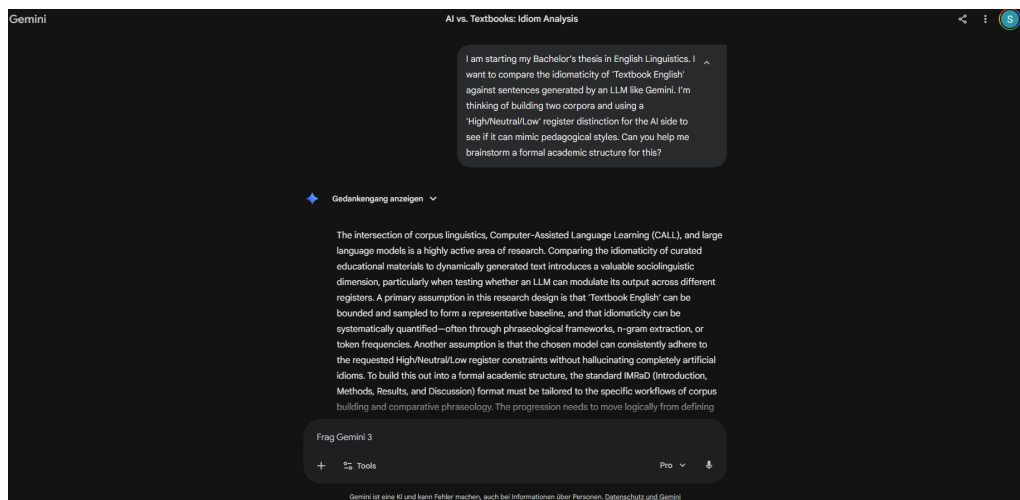


Figure 5: Gemini 3 ideation chat screenshot 1

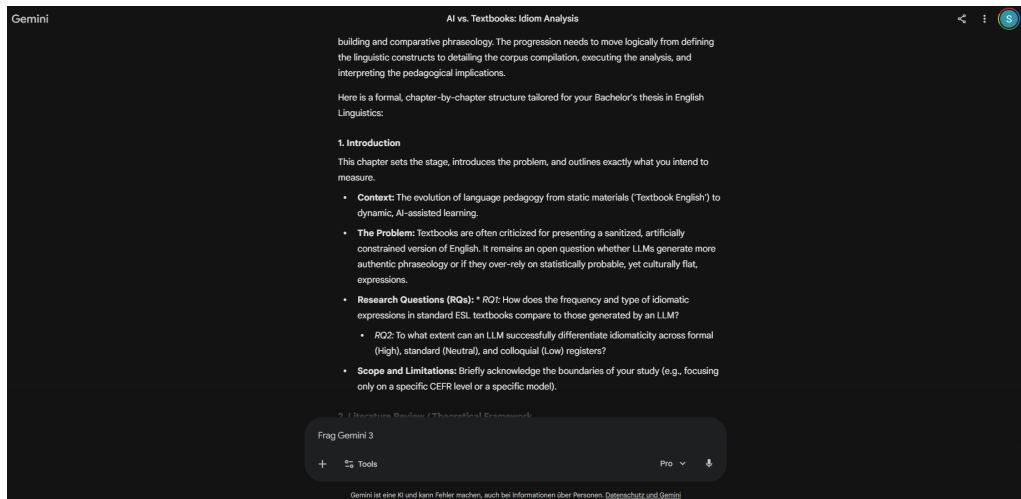


Figure 6: Gemini 3 ideation chat screenshot 2

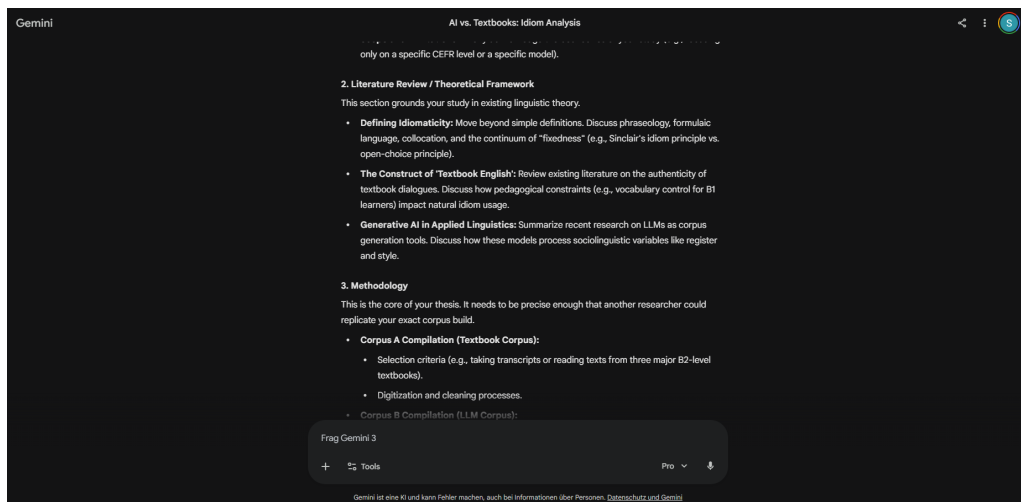


Figure 7: Gemini 3 ideation chat screenshot 3

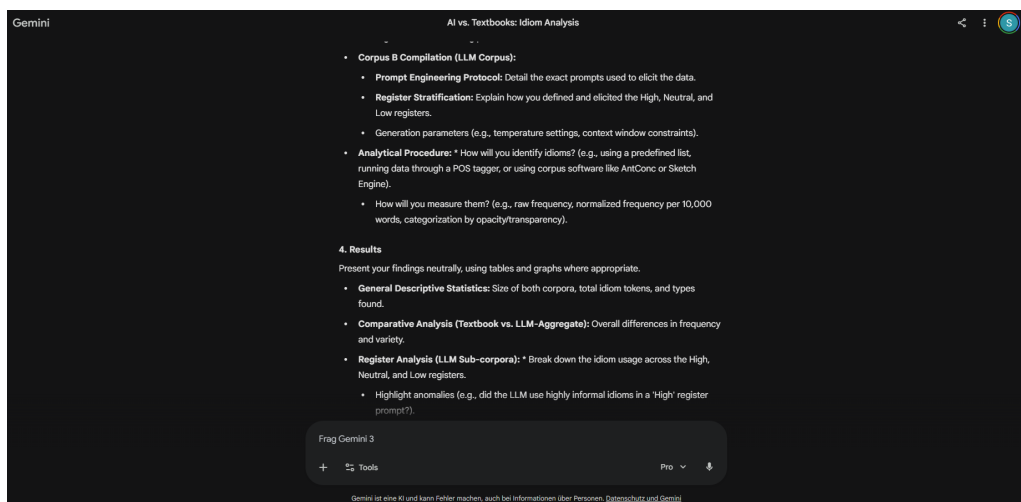


Figure 8: Gemini 3 ideation chat screenshot 4

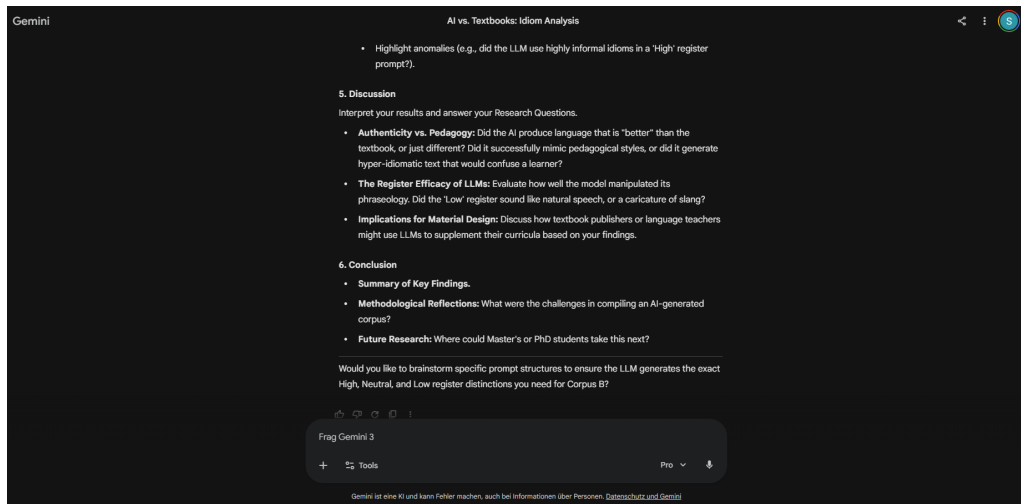


Figure 9: Gemini 3 ideation chat screenshot 5